Guia Definitivo: Executando Claude Code 100% Localmente em Apple Silicon

Autor: Manus AI **Data:** 23 de Abril de 2026

Este documento técnico detalha a arquitetura, a teoria e o passo a passo prático para executar o **Claude Code** de forma totalmente local e offline em computadores Mac com processadores Apple Silicon (M-series). Esta abordagem elimina os custos recorrentes da API da Anthropic, garantindo total privacidade dos dados e aproveitando o poder de processamento otimizado da arquitetura de memória unificada da Apple.

---

1. Fundamentos Teóricos e Arquitetura

A possibilidade de rodar o Claude Code localmente com alta performance (como os 65 tokens por segundo mencionados para modelos de 122B) baseia-se em duas inovações principais: a arquitetura de hardware da Apple e o uso de proxies de API.

1.1. A Vantagem do Apple Silicon: Memória Unificada (UMA)

O grande diferencial dos chips Apple Silicon (M2, M3, M4, M5) para inferência de Inteligência Artificial é a sua **Unified Memory Architecture (UMA)** [1]. Em PCs tradicionais, a CPU e a GPU possuem pools de memória fisicamente separados. Para inferir um modelo grande (LLM), os dados precisam trafegar pelo barramento PCIe (geralmente limitado a ~32 GB/s), o que cria um gargalo massivo, já que a inferência de LLMs é limitada pela largura de banda de memória (memory bandwidth bound), e não apenas pelo poder computacional (compute bound) [1].

No Apple Silicon, a CPU, a GPU e o Neural Engine (ANE) compartilham o mesmo pool físico de memória. Um chip como o M4 Max possui até 128GB de memória unificada com uma largura de banda de 546 GB/s [1]. Isso significa que:

1. Não há necessidade de copiar dados entre a RAM do sistema e a VRAM da placa de vídeo.
2. Modelos massivos (como um Llama 3.1 de 70B quantizado) que exigiriam múltiplas GPUs caras em um PC tradicional (pois não cabem nos 24GB de uma RTX 4090) podem ser carregados inteiramente na memória unificada de um único MacBook [1].

1.2. O Framework MLX e Quantização GGUF

Para extrair o máximo dessa arquitetura, a Apple desenvolveu o **MLX**, um framework de machine learning projetado especificamente para Apple Silicon [2]. O MLX utiliza avaliação preguiçosa (lazy evaluation) e otimiza a execução diretamente na GPU via Metal Performance Shaders (MPS), sem a sobrecarga de mover dados [1].

Além disso, o uso de formatos de quantização como o **GGUF** (especificamente K-quants como `Q4_K_M`) permite reduzir drasticamente o tamanho dos modelos na memória. Um modelo de 70 bilhões de parâmetros em FP16 exigiria cerca de 140GB de RAM. Com a quantização `Q4_K_M`, o tamanho cai para aproximadamente 40GB, mantendo uma qualidade de resposta excelente e permitindo que rode confortavelmente em um Mac com 64GB ou 128GB de RAM [1].

1.3. A Arquitetura do Proxy Local

O Claude Code foi projetado nativamente para se comunicar com a API oficial da Anthropic. O "truque" para rodá-lo localmente sem modificar o código-fonte da ferramenta é utilizar um **Proxy Server Local** [3].

A arquitetura funciona da seguinte maneira:

1. O Claude Code envia uma requisição HTTP (formato Anthropic) para `localhost`.
2. O Servidor Proxy intercepta essa requisição.
3. O Proxy traduz o payload do formato Anthropic para o formato OpenAI ou Ollama.
4. A requisição traduzida é enviada para o motor de inferência local (como o Ollama rodando um modelo Qwen ou GLM).
5. O motor local gera a resposta e a devolve ao Proxy.

6. O Proxy converte a resposta de volta para o formato Anthropic e a entrega ao Claude Code.

Repositórios como o `1rgs/claude-code-proxy` e o `mukiwu/anthropic-api-bridge` implementam exatamente essa camada de tradução, geralmente utilizando FastAPI e LiteLLM [3] [4].

---

2. Requisitos de Sistema

Para obter uma experiência fluida e produtiva, semelhante à nuvem, as seguintes especificações são recomendadas:

Componente	Requisito Mínimo	Recomendado (Para modelos > 30B)
Processador	Apple M2/M3 Pro	Apple M3/M4/M5 Max ou Ultra
Memória (RAM)	32GB Unificada	64GB a 128GB Unificada
Armazenamento	50GB de espaço livre	150GB+ de SSD NVMe
Software	macOS 14+, Python 3.12+	macOS 14+, Python 3.12+, Homebrew

---

3. Passo a Passo de Implementação

Existem duas abordagens principais para configurar o ambiente. A primeira utiliza a integração nativa do Ollama (mais simples), e a segunda utiliza um proxy dedicado (mais flexível).

Método 1: Integração Direta com Ollama (Recomendado)

O Ollama recentemente adicionou suporte nativo à API compatível com Anthropic, tornando este o método mais direto [5].

Passo 1: Instalar o Ollama Baixe e instale o Ollama para macOS a partir do site oficial ou via Homebrew:

```
brew install ollama
```

Passo 2: Baixar um Modelo Otimizado para Código Para o Claude Code, você precisa de modelos com grande janela de contexto e excelente capacidade de codificação. Recomenda-se o Qwen 2.5 Coder ou o GLM-4.

```
ollama pull qwen2.5-coder:32b
# ou para máquinas com menos RAM:
ollama pull qwen2.5-coder:7b
```

Passo 3: Instalar o Claude Code Instale a CLI oficial do Claude Code via npm:

```
npm install -g @anthropic-ai/claude-code
```

Passo 4: Configurar Variáveis de Ambiente e Executar O Claude Code permite sobrescrever a URL base da API. Aponte-a para o servidor local do Ollama (porta padrão 11434).

```
export ANTHROPIC_AUTH_TOKEN="ollama"
export ANTHROPIC_API_KEY="sk-local"
export ANTHROPIC_BASE_URL="http://localhost:11434"

claude --model qwen2.5-coder:32b
```

Nota: O Ollama não exige uma chave de API real, mas o SDK do Claude exige que a variável não esteja vazia.

Método 2: Usando o Claude Code Proxy (Para maior controle)

Se você deseja usar modelos hospedados no LM Studio ou precisa de logs detalhados da tradução, o proxy `1rgs/claude-code-proxy` é a melhor opção [3].

Passo 1: Clonar e Configurar o Proxy

```
git clone https://github.com/1rgs/claude-code-proxy.git
cd claude-code-proxy
cp .env.example .env
```

Passo 2: Editar o arquivo `.env` Abra o arquivo `.env` e configure para usar o seu provedor local (ex: Ollama ou LM Studio via formato OpenAI).

```
 PREFERRED_PROVIDER="openai"
OPENAI_BASE_URL="http://localhost:11434/v1" # URL do Ollama
OPENAI_API_KEY="sk-local"
BIG_MODEL="qwen2.5-coder:32b"
SMALL_MODEL="qwen2.5-coder:7b"
```

Passo 3: Iniciar o Servidor Proxy O projeto utiliza o gerenciador de pacotes `uv` para alta performance.

```
pip install uv
uv run uvicorn server:app --host 0.0.0.0 --port 8082
```

Passo 4: Executar o Claude Code Em outro terminal, inicie o Claude Code apontando para o seu novo proxy na porta 8082:

```
export ANTHROPIC_API_KEY="sk-local"
export ANTHROPIC_BASE_URL="http://localhost:8082"

claude
```

---

4. Bônus: Controle Remoto via iMessage / Telegram

A postagem original menciona a capacidade de controlar o Claude Code pelo iPhone enquanto o Mac processa as tarefas na mesa. Isso é possível integrando o Claude Code com bots de mensagens.

A documentação oficial do Ollama detalha a integração com o Telegram [5]. Você pode iniciar o Claude Code com a flag de canais para conectá-lo a um bot:

```
ollama launch claude -- --channels plugin:telegram@claude-plugins-official
```

Após configurar o token do bot via `@BotFather` no Telegram, você pode enviar comandos de texto do seu sofá. O Mac executará a análise de código pesada localmente e enviará a resposta de volta para o seu celular.

---

5. Conclusão

A combinação de processadores Apple Silicon (com sua arquitetura de memória unificada), frameworks de quantização eficientes e proxies de tradução de API democratizou o acesso a agentes de codificação autônomos. Executar o Claude Code localmente não é apenas uma medida de economia de custos (evitando faturas de centenas de dólares em tokens da API), mas também uma garantia absoluta de privacidade para bases de código corporativas sensíveis.

Com a chegada iminente dos chips M5, que prometem ainda mais núcleos de GPU e maior largura de banda de memória, a inferência local de modelos de 100B+ parâmetros se tornará o padrão para desenvolvedores de software avançados.

Referências

[1] Kim, Y. (2026). *Running LLMs on Apple Silicon: Inside M4/M5 Architecture for AI Inference*. Chaos and Order Blog. Disponível em: https://www.youngju.dev/blog/culture/2026-03-18-apple-silicon-llm-inference-deep-dive.en [2] Apple Machine Learning Research. (2023). *MLX: An array framework for Apple silicon*. GitHub. Disponível em: https://github.com/ml-explore/mlx [3] 1rgs. (2026). *claude-code-proxy: Run Claude Code on OpenAI models*. GitHub. Disponível em: https://github.com/1rgs/claude-code-proxy [4] mukiwu. (2026). *anthropic-api-bridge*. GitHub. Disponível em: https://github.com/mukiwu/anthropic-api-bridge [5] Ollama Documentation. (2026). *Claude Code Integration*. Disponível em: https://docs.ollama.com/integrations/claude-code